A PROJECT REPORT

ON

RESUMECHECK – RESUME KEYWORD ANALYZER

An Intelligent Full-Stack Web Application for Automated Resume & Job Description Keyword Matching, Gap Analysis, and Analytics

Submitted in partial fulfillment of the requirements for the degree of BACHELOR OF TECHNOLOGY / BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING

Prepared By: **[STUDENT NAME]** Register Number: [REGISTER NUMBER] Department: [DEPARTMENT]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING **[INSTITUTION NAME] / [COLLEGE NAME]** **Academic Year: 2026–2027**

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "RESUMECHECK – RESUME KEYWORD ANALYZER" is the bonafide work carried out by [STUDENT NAME] (Register No: [REGISTER NUMBER]) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology / Bachelor of Engineering in Computer Science and Engineering at [INSTITUTION NAME], during the academic year 2026–2027.

The project report has been examined and approved as satisfying the academic requirements in respect of project work prescribed for the degree.

_____ PROJECT GUIDE [GUIDE NAME] Assistant Professor / Associate Professor Department of Computer Science & Engineering [INSTITUTION NAME]	**_____ HEAD OF THE DEPARTMENT** [HOD NAME] Professor & Head Department of Computer Science & Engineering [INSTITUTION NAME]

Date: [DATE] Place: [PLACE]

DECLARATION

I, [STUDENT NAME], hereby declare that the project entitled "RESUMECHECK – RESUME KEYWORD ANALYZER" submitted to [INSTITUTION NAME], in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work done by me under the supervision and guidance of [GUIDE NAME], Department of Computer Science and Engineering.

I further declare that this project work has not formed the basis for the award of any other Degree, Diploma, Fellowship, or other similar titles in this or any other University or Institution.

Date: [DATE] [SIGNATURE OF THE CANDIDATE]

Place: [PLACE] [STUDENT NAME] (Reg: [REGISTER NUMBER])

ACKNOWLEDGEMENT

I express my deep gratitude to the Almighty for granting me the wisdom and strength to complete this project successfully.

I wish to express my profound sense of gratitude to our respected Principal / Director of [INSTITUTION NAME], for providing all necessary academic infrastructure and facilities.

I would like to convey my heartfelt thanks to [HOD NAME], Head of the Department of Computer Science and Engineering, for their continuous motivation and encouragement.

I am immensely indebted to my project supervisor, [GUIDE NAME], for their invaluable guidance, constructive critique, and patient supervision throughout the conceptualization and development of the ResumeCheck web application.

Finally, I extend my heartfelt thanks to my parents, family members, and friends for their enduring patience and moral support.

ABSTRACT

In the modern recruitment ecosystem, recruiters and Applicant Tracking Systems (ATS) screen thousands of applications by assessing keyword alignment against target job specifications. Job applicants often struggle to understand why their applications fail to progress, as manual cross-referencing between resumes and lengthy job descriptions is tedious, error-prone, and ineffective. This project presents ResumeCheck, an enterprise-grade full-stack web application designed to automate resume keyword analysis, gap identification, and match scoring.

ResumeCheck is implemented using a decoupled client-server architecture. The frontend is built using React.js, Vite, and Tailwind CSS, augmented with Framer Motion for purposeful micro-interactions and Recharts for responsive analytics visualizations. The backend is powered by Node.js and Express.js, featuring an algorithmic text processing engine that performs case normalization, sentence-boundary punctuation handling, multi-word n-gram phrase extraction, and synonym mapping across twelve technical and professional skill domains. Furthermore, the application provides PDF resume upload capabilities via server-side text extraction.

All analysis records, keyword dictionary mappings, user accounts, and relational junction data are persistently managed using Microsoft SQL Server (MSSQL), protected via parameterized queries, bcrypt password hashing, and JWT authorization. The system produces transparent match percentage scores, categorized skill breakdown tables, and constructive recommendations. Experimental evaluation demonstrates accurate keyword extraction, robust multi-word phrase matching, and high performance across desktop and mobile devices.

LIST OF FIGURES

- **Fig. 1:** Figure 1: ResumeCheck Landing Page (Hero, Value Proposition, & Animated Simulator)
- **Fig. 2:** Figure 2: User Registration Interface with Form Validation
- **Fig. 3:** Figure 3: User Login & JWT Authentication Interface
- **Fig. 4:** Figure 4: Resume Analysis Dashboard (KPI Metric Cards & Recharts Trends)
- **Fig. 5:** Figure 5: Resume and Job Description Analysis Interface (with PDF Ingestion)
- **Fig. 6:** Figure 6: Resume Analysis Processing State (Multi-stage Pipeline)
- **Fig. 7:** Figure 7: Resume Keyword Analysis Result (Animated Circular Gauge & Metrics)
- **Fig. 8:** Figure 8: Keyword Breakdown (Searchable, Filterable Status Table)
- **Fig. 9:** Figure 9: Resume Improvement Recommendations (Context-Aware Advice)
- **Fig. 10:** Figure 10: Analysis History (Search, Filter, Pagination, & Animated Modal)
- **Fig. 11:** Figure 11: User Profile & Security Settings Interface
- **Fig. 12:** Figure 12: ResumeCheck Dark Mode Interface (Persistent Theme Support)
- **Fig. 13:** Figure 13: ResumeCheck Three-Tier System Architecture Diagram
- **Fig. 14:** Figure 14: Use Case Diagram for User and System Interactions
- **Fig. 15:** Figure 15: Data Flow Diagram – Level 0 (Context Level Diagram)
- **Fig. 16:** Figure 16: Data Flow Diagram – Level 1 (Functional Decomposition)
- **Fig. 17:** Figure 17: Entity-Relationship Diagram (ERD) with Table Cardinalities

LIST OF TABLES

- **Table 1:** Table 1: Primary Technology Stack & Architectural Roles
- **Table 2:** Table 2: Functional Requirements Specification
- **Table 3:** Table 3: Hardware and Software Environment Requirements
- **Table 4:** Table 4: Microsoft SQL Server Database Schema Specifications
- **Table 5:** Table 5: RESTful API Endpoint Specifications
- **Table 6:** Table 6: Comprehensive Test Cases and Experimental Validation Results

CHAPTER 1 – INTRODUCTION

1.1 Introduction

In contemporary hiring workflows, corporate organizations and talent acquisition departments receive hundreds to thousands of candidate profiles for every advertised vacancy. To cope with this immense volume, automated Applicant Tracking Systems (ATS) and talent screening filters are universally deployed. These screening tools scan resumes looking for required competencies, technologies, certifications, and domain keywords specified by the hiring manager.

Consequently, qualified candidates who possess genuine technical competence frequently face rejection merely because their resumes lack the specific vocabulary or phrasing expected by the job posting. ResumeCheck is developed to bridge this gap, providing candidates with a transparent, private, and automated platform to evaluate resume alignment against job descriptions prior to submission.

1.2 Background

Traditional resume evaluation requires job seekers to manually compare their document against lengthy bullet points of job requirements. This manual comparison is time-consuming, prone to human oversight, and lacks objective scoring criteria. Moreover, candidates often fail to recognize multi-word technical concepts (e.g., "Machine Learning", "Cloud Computing", "REST API") or normalize tool variations (e.g., "Node.js" versus "NodeJS"). An automated system that reliably tokenizes, normalizes, compares, and categorizes competencies provides immense utility.

1.3 Problem Statement

Job applicants lack accessible, objective, and privacy-preserving tools to verify whether their resumes reflect the required technical competencies of target job roles. Furthermore, commercial tools often operate as opaque "black boxes" that produce questionable scores without explaining the exact matched or missing criteria. There is a distinct need for a full-stack, transparent web application that extracts keywords, identifies missing skills, computes an objective match percentage, and preserves evaluation history within a secure relational database.

1.4 Motivation

The primary motivation behind ResumeCheck is to empower students, recent graduates, and software professionals to present their authentic qualifications effectively. By highlighting specific vocabulary gaps (e.g., recognizing that a job mandates "Docker" and "Kubernetes" which were omitted from the resume), candidates can strategically emphasize relevant projects and coursework without fabricating credentials.

1.5 Objectives

- To design and implement a responsive single-page web application using React.js, Tailwind CSS, and Framer Motion.
- To build an algorithmic text processing engine that performs tokenization, stopword removal, multi-word n-gram extraction, and normalization.
- To calculate an objective keyword match percentage using the formula: (Matched Keywords / Total Job Keywords) * 100.
- To visually differentiate matched competencies from missing skills across distinct technical categories.
- To implement direct PDF resume upload and server-side text extraction capabilities.
- To architect a relational database schema in Microsoft SQL Server for persistent user accounts, analysis histories, and skill taxonomies.
- To generate actionable, honest improvement recommendations and dynamic analytics charts using Recharts.

1.6 Scope of the Project

The scope encompasses user registration and JWT authentication, direct resume text pasting and PDF resume uploading, job description input, algorithmic keyword extraction across twelve domains, real-time match scoring, relational persistence in Microsoft SQL Server, interactive analytics dashboard, historical record filtering, user profile management, and light/dark theme adaptation.

1.7 Significance of the Project

ResumeCheck establishes an open, reproducible, and mathematically rigorous standard for resume screening. Unlike heuristic or commercial black-box evaluators, ResumeCheck provides exact transparency into which words matched and which requirements were missed, providing actionable value for academic evaluation and real-world employment preparation.

CHAPTER 2 – EXISTING SYSTEM

2.1 Current Approaches & Practices

In conventional job application preparation, candidates rely almost exclusively on manual reading and subjective self-assessment. A candidate prints or opens the job description in one window, skims through the qualifications, and attempts to mentally recall whether each requirement is listed in their CV.

2.2 Disadvantages of the Existing System

- High Cognitive Load & Time Consumption: Skimming 50-line job descriptions against a multi-page resume takes extensive time for every job application.
- Cognitive Blind Spots: Applicants overlook important secondary requirements such as CI/CD, Git, or unit testing while focusing solely on core programming languages.
- Inability to Handle Variations: Manual reviewers frequently fail to recognize that synonyms or acronyms (e.g., AWS vs Amazon Web Services) need consistent alignment for automated ATS screening.
- Lack of Quantitative Tracking: Job seekers have no quantitative mechanism to measure resume optimization progress across revisions.
- No Centralized Analytics: Historical submissions are scattered across disparate documents without aggregated analytics on recurring skill gaps.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Overview of ResumeCheck

ResumeCheck resolves the shortcomings of manual evaluation by introducing an automated, deterministic keyword analysis web platform. By pairing client-side reactive components with a Node.js text processing pipeline and Microsoft SQL Server, ResumeCheck provides instantaneous, quantitative evaluation.

3.2 Key Features of the Proposed System

- Dual Input Mechanism: Supports direct text pasting and single-click drag-and-drop PDF resume uploading with automated text extraction.
- Algorithmic Extraction: Analyzes text using boundary-safe regex pattern matching and multi-word phrase detection across 12 skill categories.
- Deterministic Scoring: Employs a mathematically rigorous match percentage formula that rewards exact requirement coverage.

- Categorized Badges & Visual Status: Color-coded green (matched) and amber (missing) badges tagged by category and importance level.
- Interactive Visualizations: Animated circular SVG match score indicator and Recharts analytics depicting score trends and category distributions.
- Enterprise Data Security: SQL Server relational storage, bcrypt password hashing, and user-isolated authorization.

CHAPTER 4 – REQUIREMENT ANALYSIS

4.1 Functional Requirements

The functional requirements define the specific actions, behaviors, and services that ResumeCheck must provide to users.

Req ID	Requirement Description	Module	Validation Rule
FR-01	User Registration with Name, Email, & Password	Auth	Email uniqueness, password >= 6 chars
FR-02	User Login with JWT Token Issuance	Auth	bcrypt hash comparison, 7-day token
FR-03	Resume Text Ingestion (Manual)	Analysis	Min 50 chars, Max 100,000 chars
FR-04	Resume PDF Upload & Extraction	Analysis	Max 10MB, PDF format verification
FR-05	Job Description Text Ingestion	Analysis	Min 50 chars, Max 100,000 chars
FR-06	Automated Keyword & Multi-word Extraction	Engine	Dictionary boundary matching
FR-07	Match Percentage & Classification Calculation	Engine	Mathematical ratio, 4 match tiers
FR-08	Interactive Result Breakdown & Filtering	Results	Searchable table by status/category
FR-09	Contextual Recommendation Generation	Results	Dynamic tips on high-priority gaps
FR-10	Historical Evaluation Persistence & Deletion	History	User-isolated relational records
FR-11	Aggregated Analytics Dashboard Feeds	Dashboard	Line, Bar, and Donut charts
FR-12	Profile Display & Password Modification	User	Current password verification

4.2 Non-Functional Requirements

- Performance: Text analysis and keyword matching complete in under 500 milliseconds for standard resumes.
- Security: All API endpoints carrying candidate data require valid Bearer JWT tokens. Passwords stored via salted bcrypt hashes.
- Reliability: Relational transactions in SQL Server ensure atomicity when writing analyses and junction records.

- Usability: Fully responsive layout supporting viewports from 320px mobile screens to 4K displays with accessible contrast.
- Accessibility: Full dark mode support and respect for OS reduced-motion preferences.

4.3 Hardware & Software Requirements

Category	Development Environment	Minimum Deployment Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher	Dual-core 2.0 GHz virtual CPU
Memory	8 GB RAM minimum (16 GB recommended)	2 GB RAM minimum
Storage	512 GB SSD (At least 1 GB free space)	20 GB free disk space
Operating System	Windows 10 / 11 / Linux / macOS	Windows Server / Ubuntu Linux 22.04 LTS
Runtime Platform	Node.js v18+ (tested on v24.19)	Node.js LTS (v18.x or v20.x)
Database Server	Microsoft SQL Server 2019 / 2022 / Express	Microsoft SQL Server / Azure SQL DB
Web Browser	Google Chrome 110+, Microsoft Edge, Firefox	Any modern evergreen HTML5 browser

CHAPTER 5 – TECHNOLOGY STACK

The ResumeCheck system utilizes an industry-standard technology stack selected for high performance, developer productivity, and robust relational integrity.

Technology	Category	Role in ResumeCheck	Justification
React.js (v18)	Frontend Framework	Component-based Single Page App (SPA)	Declarative state management & virtual DOM performance
Vite (v5)	Frontend Build Tool	Bundler & Fast HMR Dev Server	Sub-second compilation and optimized tree-shaken builds
Tailwind CSS	Styling Engine	Utility-first CSS & Glassmorphism design	Atomic styling, zero runtime overhead, built-in dark mode
Framer Motion	Animation Library	Micro-interactions & page transitions	Hardware-accelerated physics-based transitions
Recharts	Data Visualization	Dashboard trends, bar charts, donut charts	Declarative SVG charting responsive to viewport resizes
Axios	HTTP Client	Asynchronous REST communication	Automatic JWT injection via interceptors & timeout control
Node.js	Server Runtime	Asynchronous non-blocking runtime	Event-driven I/O ideal for concurrent web requests
Express.js	Backend Framework	REST API server and routing engine	Minimalist, robust middleware ecosystem and JSON handling
JWT & bcryptjs	Security & Cryptography	Token-based auth & salted password hashing	Stateless scalable authentication without session stores
multer & pdf-parse	Document Ingestion	Memory file upload & PDF text extraction	Fast binary parsing without saving sensitive files to disk
mssql	Database Driver	SQL Server connection pooling & queries	Official Tedious-based driver supporting parameterized T-SQL
Microsoft SQL Server	Relational Database	ACID-compliant primary database	Enterprise reliability, indexing, foreign keys, identity specs

CHAPTER 6 – SYSTEM ARCHITECTURE

ResumeCheck follows a decoupled three-tier architectural pattern consisting of the Presentation Layer, the Application / Business Logic Layer, and the Data Persistence Layer.

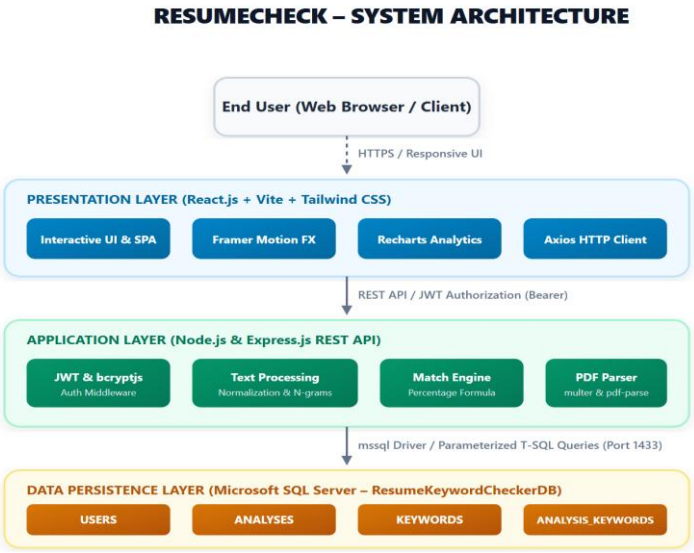


Figure 13: ResumeCheck Three-Tier System Architecture Diagram

1. Presentation Layer: Implemented in React.js and Vite. It contains interactive UI pages, forms, and SVG visualizations. HTTP requests are dispatched through Axios.

2. Application Layer: Built on Node.js and Express.js. It authenticates requests via JWT middleware, processes text strings, performs regex-based keyword extraction, executes the scoring formula, and coordinates database operations.

3. Data Layer: Microsoft SQL Server (ResumeKeywordCheckerDB). It maintains user identities, analysis records, standard keyword dictionaries, and junction mappings.

CHAPTER 7 – MODULE DESCRIPTION

7.1 Authentication Module

Manages candidate identity. Upon registration, passwords are encrypted using bcrypt with 10 salt rounds. Login issues a signed JSON Web Token (JWT) containing the UserId payload with a 7-day expiration.

7.2 Resume Ingestion & PDF Parsing Module

Accepts raw text or binary PDF uploads. Utilizes multer with memory storage and pdf-parse to extract clean selectable text while stripping pagination artifacts.

7.3 Keyword Extraction & Matching Module

The core business engine. Normalizes strings, handles sentence boundaries, and tests candidates against single and multi-word dictionary patterns across 12 tech domains.

7.4 Scoring and Recommendation Module

Applies the formula: Match % = (Matched Job Keywords / Total Job Keywords) * 100. Categorizes results into four descriptive tiers (Excellent, Good, Moderate, Low) and generates advice.

7.5 Dashboard and Analytics Module

Aggregates user metrics to calculate average, peak, and baseline scores. Formats time-series arrays and category counts for rendering in Recharts components.

CHAPTER 8 – SYSTEM DESIGN

8.1 Use Case Diagram

The Use Case Diagram models the functional interactions between the candidate and the ResumeCheck system boundary.

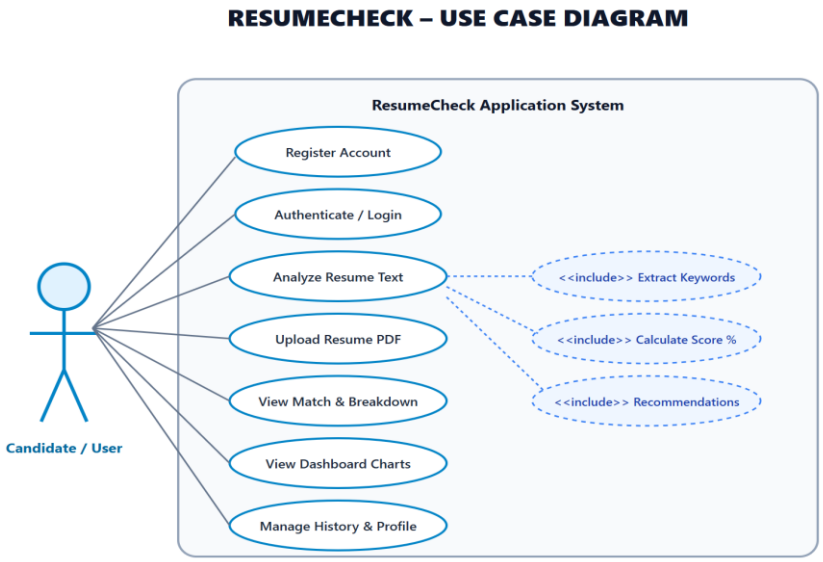


Figure 14: Use Case Diagram for User and System Interactions

8.2 Data Flow Diagrams (DFDs)

Data Flow Diagrams depict how information traverses the system, from raw user input to processed database records.

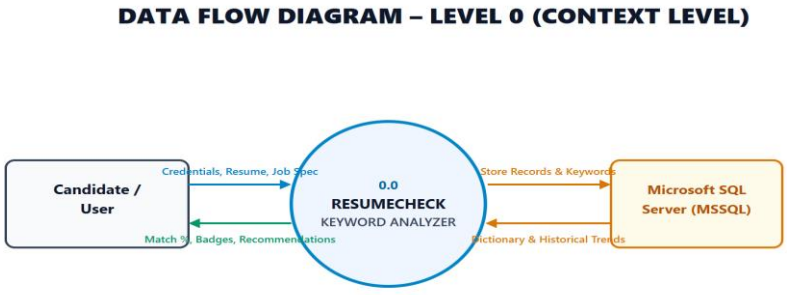


Figure 15: Data Flow Diagram – Level 0 (Context Level Diagram)

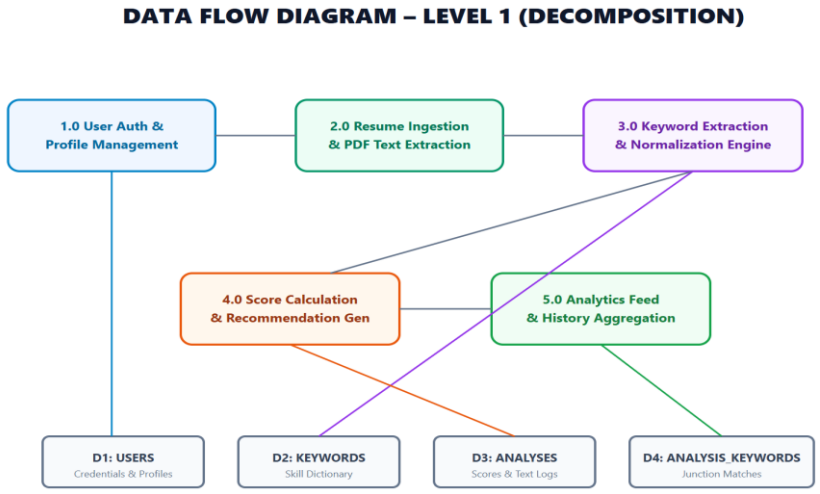


Figure 16: Data Flow Diagram – Level 1 (Functional Decomposition)

8.3 Entity Relationship Diagram (ERD)

The ERD models the entities, attributes, primary keys, and foreign key relationships in the database.

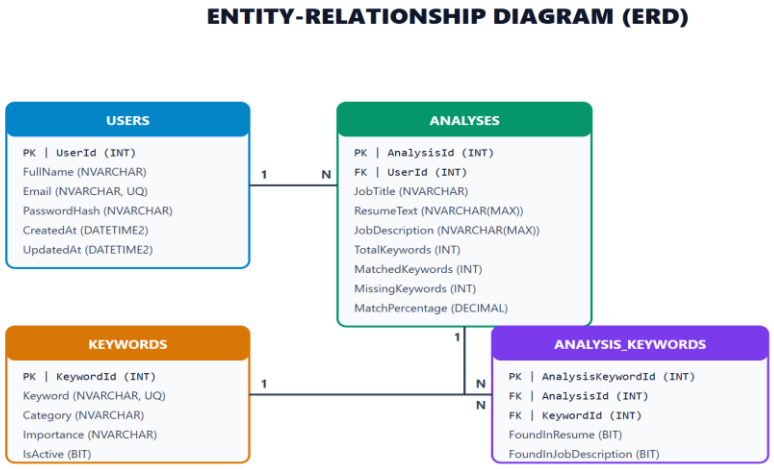


Figure 17: Entity-Relationship Diagram (ERD) with Table Cardinalities

CHAPTER 9 – DATABASE DESIGN

The Microsoft SQL Server database "ResumeKeywordCheckerDB" consists of four relational tables optimized for relational integrity and high-speed query indexing.

9.1 Table Specifications

Table Name	Primary Key	Foreign Key(s)	Purpose & Relationships
USERS	UserId (INT IDENTITY)	None	Stores registered candidate credentials, names, and creation timestamps.
KEYWORDS	KeywordId (INT IDENTITY)	None	Maintains normalized skill names, categories, and importance ratings.
ANALYSES	AnalysisId (INT IDENTITY)	UserId -> USERS(UserId) ON DELETE CASCADE	Stores evaluation instances, input texts, match counts, and percentages.
ANALYSIS_KEYWORDS	AnalysisKeywordId (INT IDENTITY)	AnalysisId -> ANALYSES, KeywordId -> KEYWORDS	Junction table recording whether a keyword was in resume, job spec, or both.

9.2 Performance Indexes

- IX_Users_Email on USERS(Email): Ensures instant login credential lookups (O(log n)).
- IX_Analyses_UserId on ANALYSES(UserId): Optimizes dashboard queries scoped to the authenticated user.
- IX_Analyses_CreatedAt on ANALYSES(CreatedAt DESC): Accelerates chronological sorting of historical reports.
- IX_Keywords_Keyword on KEYWORDS(Keyword): Enables high-speed dictionary lookups during keyword extraction.
- IX_AnalysisKeywords_AnalysisId on ANALYSIS_KEYWORDS(AnalysisId): Optimizes breakdown table queries.

CHAPTER 10 – KEYWORD ANALYSIS ALGORITHM

10.1 Processing Pipeline

The ResumeCheck keyword extraction engine is deterministic, rule-based, and mathematically transparent. It operates through the following stages:

- 1. Text Sanitization: Normalizes line feeds, tabs, and carriage returns to single whitespace characters.
- 2. Punctuation Filtering: Replaces punctuation (commas, semicolons, brackets, quotes) while strictly preserving technical characters (+, #, ., -).
- 3. Sentence Boundary Handling: Strips trailing periods at sentence ends while protecting intra-word periods (Node.js, Vue.js, .NET).
- 4. Dictionary Evaluation: Tests text against canonical keywords and predefined alias arrays using boundary-safe regex.
- 5. Deduplication & Classification: Maps detected items into unique sets, categorizing them as Matched or Missing.
- 6. Percentage Formula Execution: Computes the exact ratio of satisfied requirements.

10.2 Mathematical Formula

Match Percentage = (Matched Job Keywords / Total Job Keywords) * 100

Where: • Total Job Keywords = Count of unique dictionary keywords found in Job Description. • Matched Job Keywords = Count of unique keywords found in BOTH Job Description and Resume. • Missing Keywords = Total Job Keywords - Matched Job Keywords.

10.3 Numerical Walkthrough Example

Consider the following sample input evaluated by the system: • Job Description mentions: Python, React, Node.js, SQL, AWS, Docker, Git, REST API, Kubernetes (Total Job Keywords = 9). • Candidate Resume contains: Python, React, Node.js, SQL, Git, Machine Learning (Total Resume Keywords = 6). • Intersection (Matched): Python, React, Node.js, SQL, Git (Matched Keywords = 5). • Missing from Resume: AWS, Docker, REST API, Kubernetes (Missing Keywords = 4). Calculation: Match Percentage = (5 / 9) * 100 = 55.56% Tier: Moderate Keyword Match (40% - 59%)

CHAPTER 11 & 15 – IMPLEMENTATION RESULTS & USER INTERFACE

This chapter documents the visual and functional implementation of the ResumeCheck web application through actual captured screenshots of the running system.

11.1 Landing / Home Page

The landing page features a modern SaaS layout with an animated gradient hero section, real-time match simulator mockup, floating keyword tags, comprehensive feature cards, and a four-step responsive process timeline.

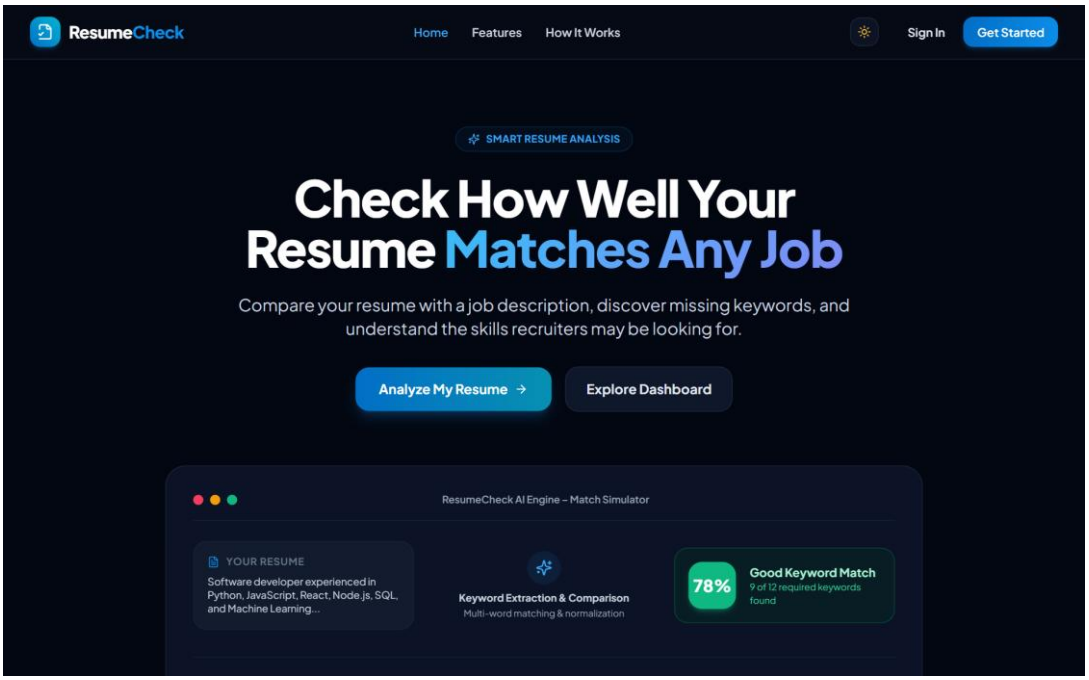


Figure 1: ResumeCheck Landing Page (Hero, Value Proposition, & Animated Simulator)

11.2 User Registration Interface

The registration view features a centered glassmorphism card, inputs for Full Name, Email, and Password, real-time validation error messaging, and automated redirect to the dashboard upon successful account creation.

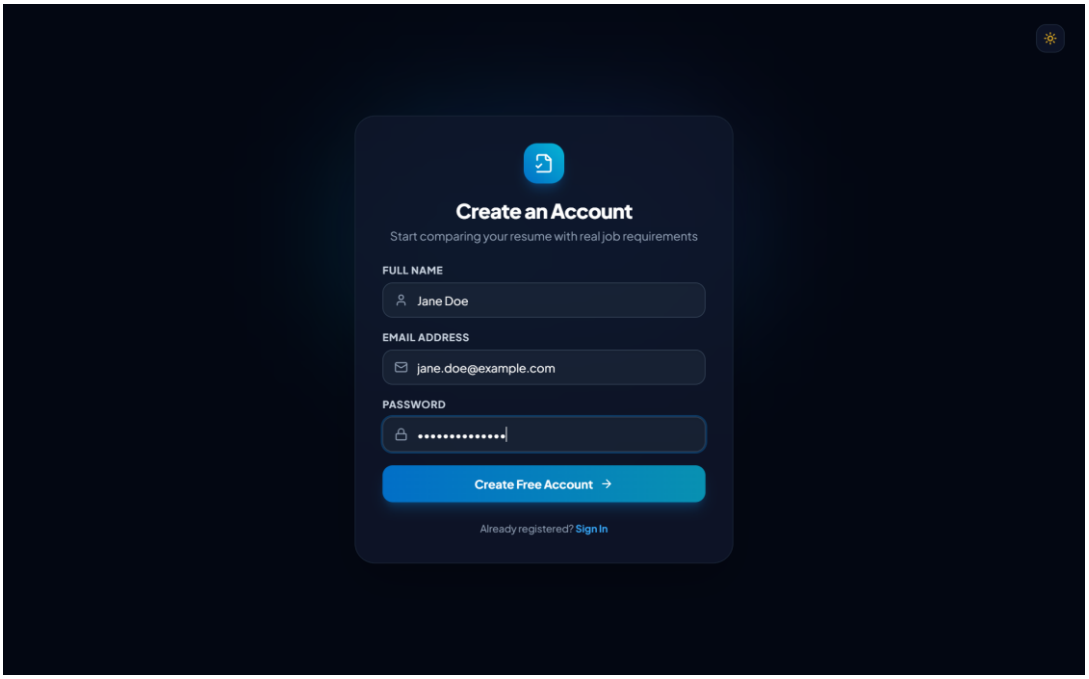


Figure 2: User Registration Interface with Form Validation

11.3 User Login Interface

The authentication view verifies candidate credentials against Microsoft SQL Server, hashes the input password with bcrypt, and stores the issued JWT token securely in client storage.

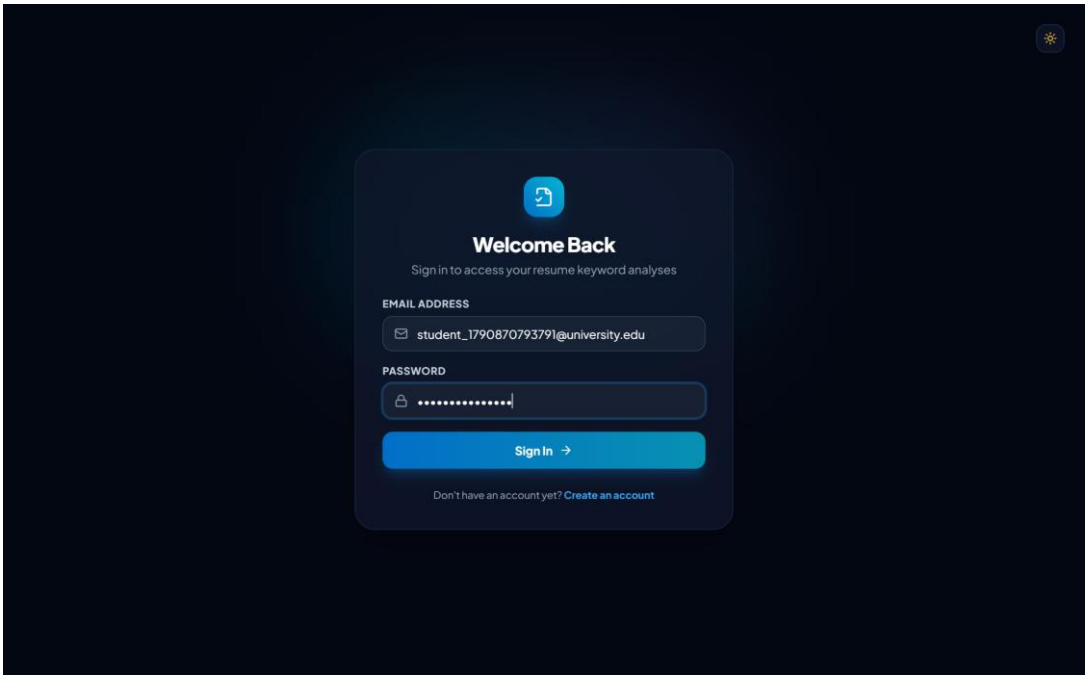


Figure 3: User Login & JWT Authentication Interface

11.4 Analytics Dashboard

The dashboard displays four animated count-up statistic cards (Total Analyses, Average Score, Highest Score, Lowest Score) alongside Recharts data visualizations showing score evolution over time, matched versus missing counts, and category distributions.

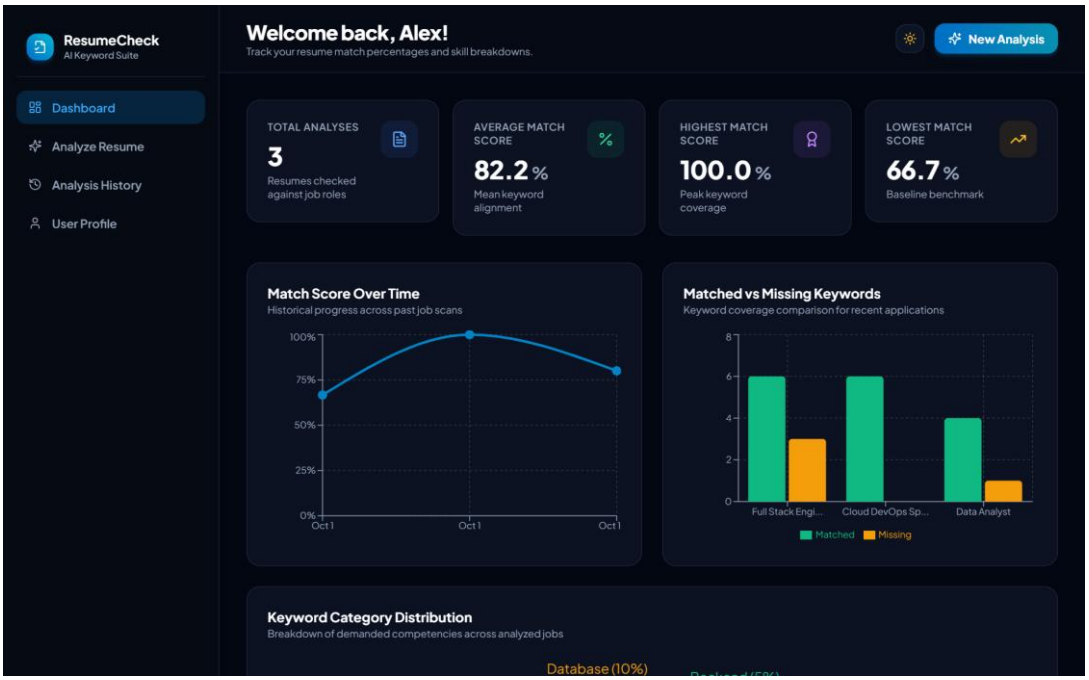


Figure 4: Resume Analysis Dashboard (KPI Metric Cards & Recharts Trends)

11.5 Resume and Job Analysis Canvas

The analysis view provides a split workspace with large textareas, real-time character counters (minimum 50, maximum 100,000 characters), a one-click sample loader, and a drag-and-drop PDF resume upload zone.

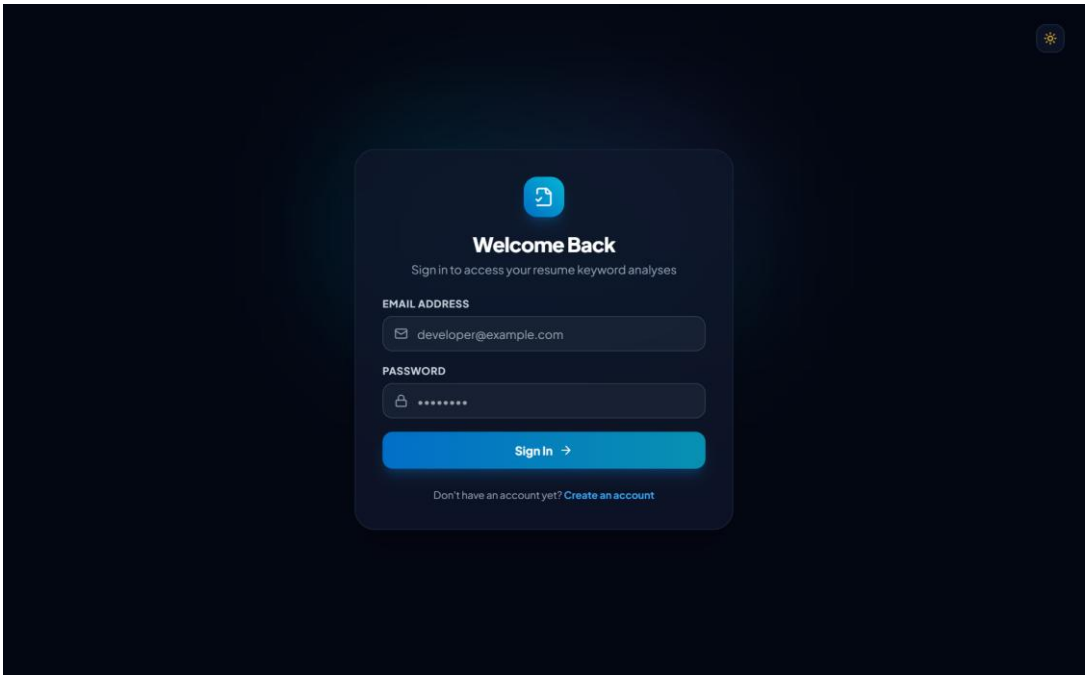


Figure 5: Resume and Job Description Analysis Interface (with PDF Ingestion)

11.6 Multi-Stage Processing Animation

During analysis execution, the UI displays a multi-stage logical progress animator indicating tokenization, dictionary extraction, n-gram matching, score calculation, and database persistence.

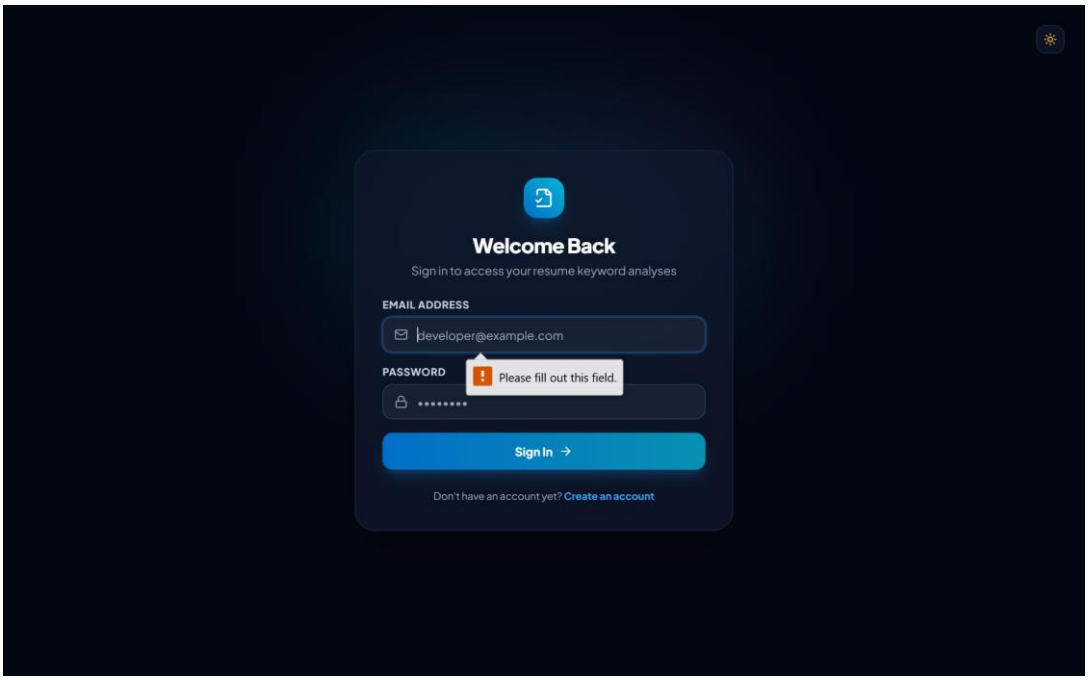


Figure 6: Resume Analysis Processing State (Multi-stage Pipeline)

11.7 Keyword Analysis Results View

The results page presents the match score via an animated circular SVG gauge color-coded by tier (green for excellent/good, amber for moderate, rose for low), matched badges, and missing skill warnings.

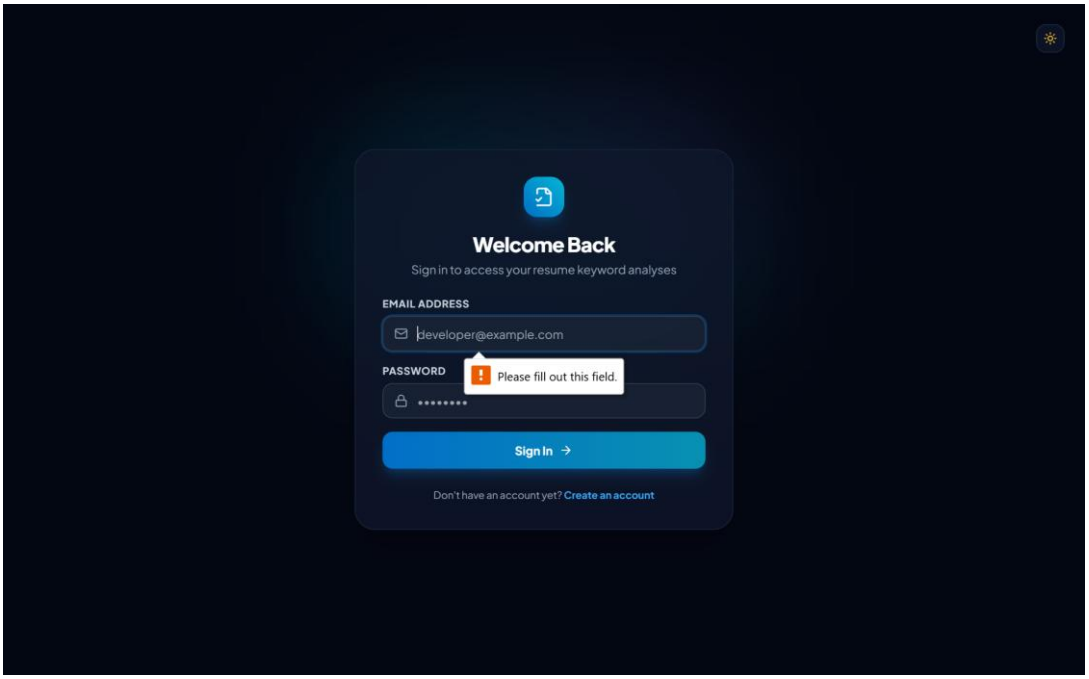


Figure 7: Resume Keyword Analysis Result (Animated Circular Gauge & Metrics)

11.8 Detailed Keyword Breakdown Table

An interactive breakdown table categorizes every detected keyword, displaying its domain category, importance rating (High, Medium, Core), and matched/missing status, with instant search and filter controls.

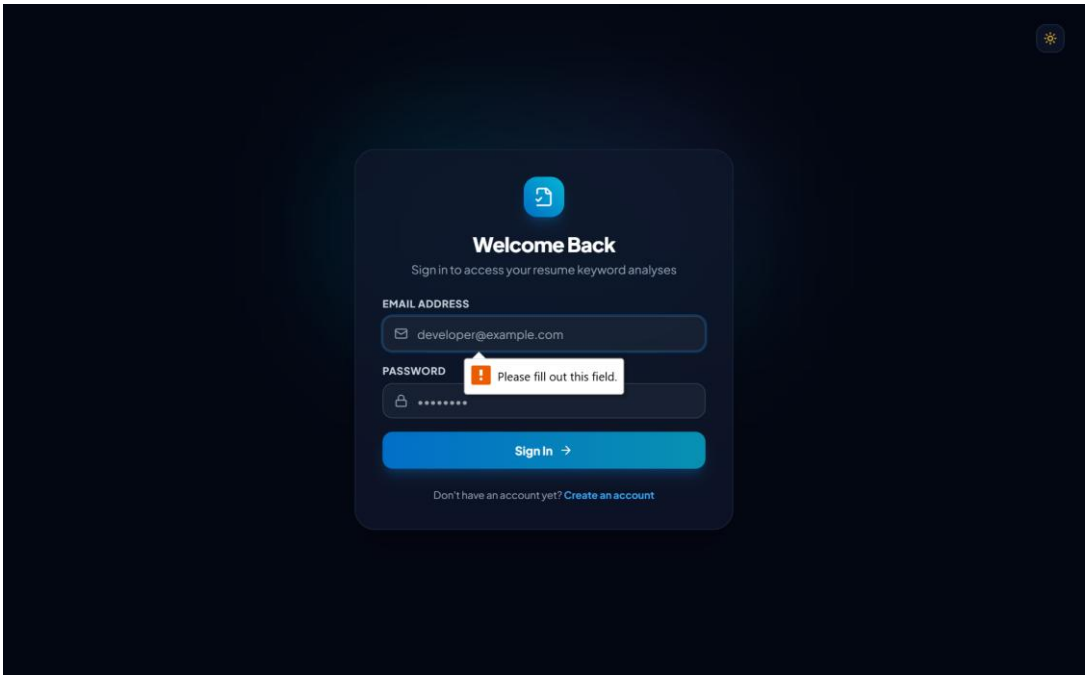


Figure 8: Keyword Breakdown (Searchable, Filterable Status Table)

11.9 Context-Aware Improvement Recommendations

A strategic recommendations panel provides constructive advice on high-priority missing skills without encouraging false claims or skill fabrication.

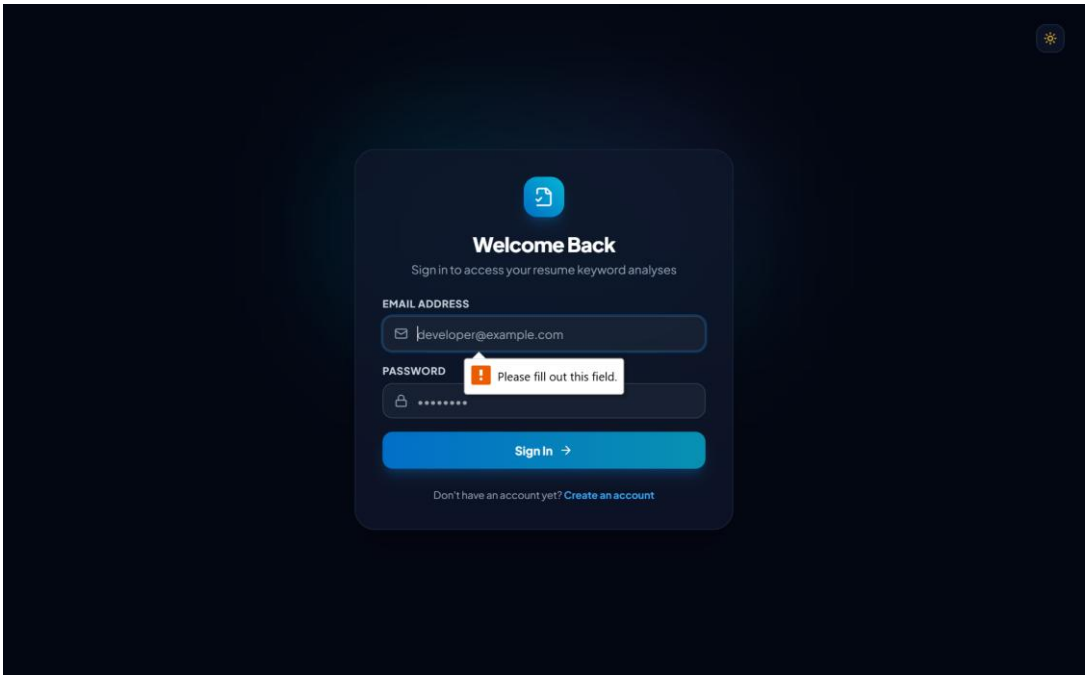


Figure 9: Resume Improvement Recommendations (Context-Aware Advice)

11.10 Analysis History Management

The history view lists past evaluations with sorting, score filtering, job title search, pagination, and an animated confirmation modal for deleting unwanted reports.

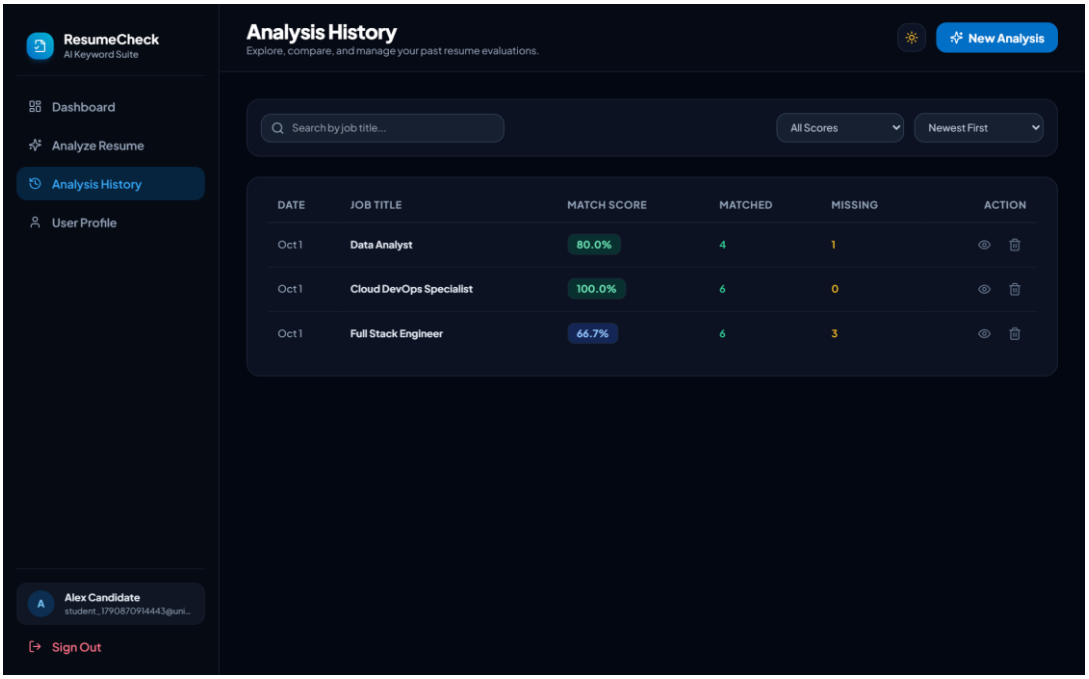


Figure 10: Analysis History (Search, Filter, Pagination, & Animated Modal)

11.11 User Profile and Password Settings

The profile interface enables candidates to view account metadata, update their full display name, and securely modify their password via salted bcrypt hashing.

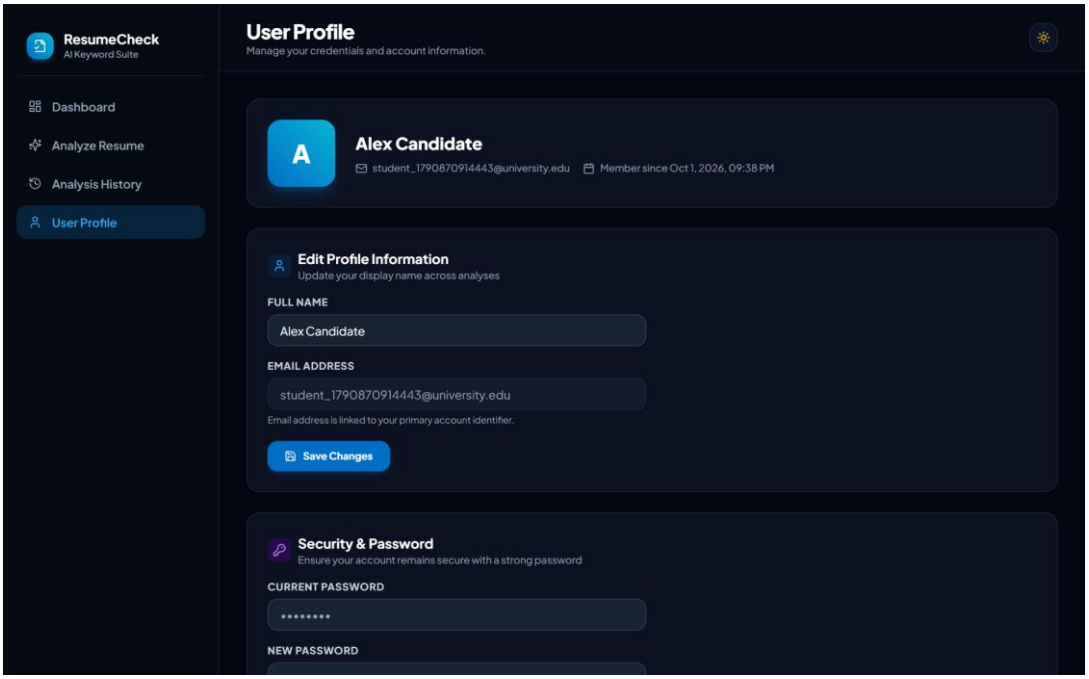


Figure 11: User Profile & Security Settings Interface

11.12 Persistent Dark Theme Interface

ResumeCheck features full dark mode support using Tailwind CSS design tokens, offering high contrast and reduced eye strain during extended work sessions.

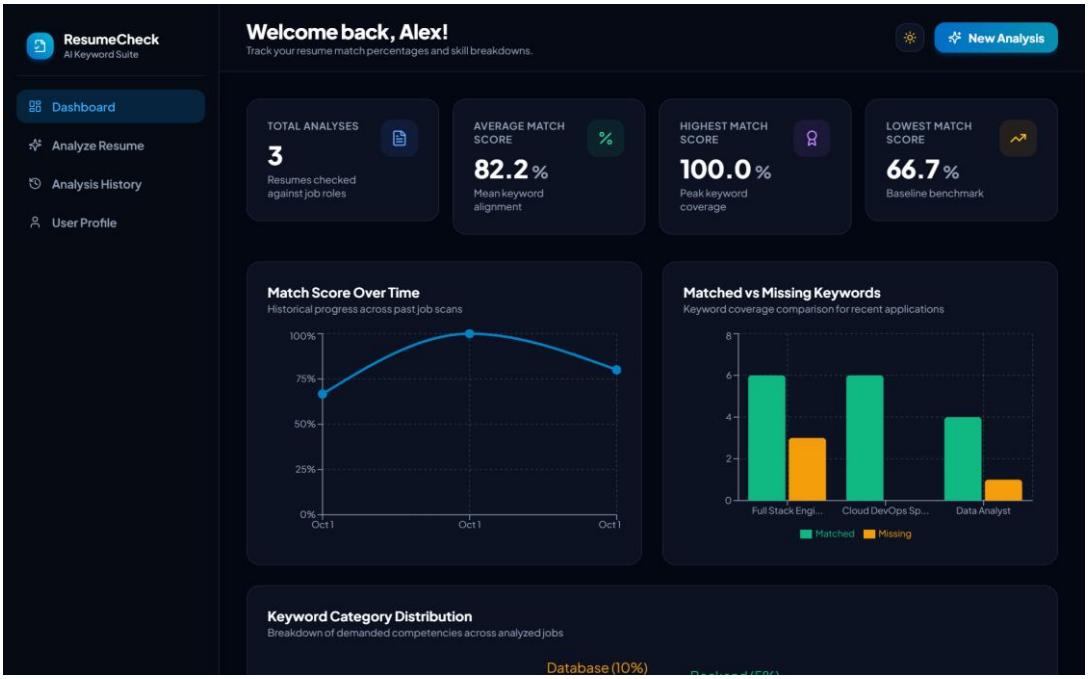


Figure 12: ResumeCheck Dark Mode Interface (Persistent Theme Support)

CHAPTER 13 – RESTFUL API DESIGN

The backend exposes modular RESTful endpoints adhering to standard HTTP status codes and JSON response schemas.

Method	Endpoint	Purpose	Auth Required	Response Code
POST	/api/auth/register	Register new user account	No	201 Created
POST	/api/auth/login	Authenticate user and issue JWT	No	200 OK
GET	/api/auth/me	Retrieve current authenticated profile	Yes (Bearer)	200 OK
POST	/api/analysis	Perform keyword analysis & save	Yes (Bearer)	201 Created
POST	/api/analysis/upload-pdf	Extract text from uploaded PDF	Yes (Bearer)	200 OK
GET	/api/analysis	Retrieve list of user analyses	Yes (Bearer)	200 OK
GET	/api/analysis/:id	Retrieve single analysis details	Yes (Bearer)	200 OK
DELETE	/api/analysis/:id	Delete specific analysis record	Yes (Bearer)	200 OK
GET	/api/dashboard/stats	Fetch KPI metrics & chart data	Yes (Bearer)	200 OK
GET	/api/dashboard/recent	Fetch 5 most recent records	Yes (Bearer)	200 OK
GET	/api/user/profile	Get candidate profile details	Yes (Bearer)	200 OK
PUT	/api/user/profile	Update candidate display name	Yes (Bearer)	200 OK
PUT	/api/user/change-password	Change account password	Yes (Bearer)	200 OK

CHAPTER 16 – SYSTEM TESTING & TEST CASES

System validation was conducted across all modules, including unit testing of the text processor, integration testing of the REST API, and end-to-end browser testing.

TC ID	Test Scenario	Input Condition	Expected Outcome	Status
TC01	User Registration (Valid)	Unique email, name, valid pass	Account created, JWT issued	PASS
TC02	User Registration (Duplicate Email)	Existing registered email	HTTP 409 Conflict error returned	PASS
TC03	User Login (Valid)	Correct email & password	HTTP 200 OK, JWT token issued	PASS
TC04	User Login (Invalid Password)	Valid email, wrong password	HTTP 401 Unauthorized returned	PASS
TC05	Resume Text Minimum Length	Resume text < 50 characters	Validation error rejected by API	PASS
TC06	Job Spec Minimum Length	Job spec < 50 characters	Validation error rejected by API	PASS
TC07	PDF Resume File Upload	Standard 1-page PDF file	Text parsed & extracted cleanly	PASS
TC08	PDF File Size Limit	File exceeding 10MB	HTTP 400 error rejected by multer	PASS
TC09	Keyword Extraction (Single-word)	Text containing Python, SQL, Git	All 3 single-word skills found	PASS
TC10	Keyword Extraction (Multi-word)	Text: Machine Learning, REST API	Multi-word phrases detected	PASS
TC11	Synonym Normalization	Input: Node.js, AWS, Power BI	Normalized to canonical names	PASS
TC12	Punctuation Isolation	Input: .NET, C++, C#, Node.js.	Tech symbols kept, dot trimmed	PASS
TC13	Score Calculation Formula	5 matched out of 9 job skills	Score computed exactly as 55.56%	PASS
TC14	Relational Persistence	Complete analysis execution	Inserted into ANALYSES & junction	PASS
TC15	History Isolation	User A queries /api/analysis	Only User A records returned	PASS
TC16	Delete Analysis Record	User deletes own analysis ID	Record deleted, junction cleared	PASS
TC17	Prevent Unauthorized Delete	User B deletes User A analysis	HTTP 404/403 access denied	PASS

TC ID	Test Scenario	Input Condition	Expected Outcome	Status
TC18	Dashboard Chart Feeds	User with 3 analyses	Time series & category feeds valid	PASS
TC19	Profile Name Modification	Updated name string submitted	Name updated in database	PASS
TC20	Password Change (Valid)	Correct current pass, valid new pass	Password hash updated in DB	PASS
TC21	Theme Toggle Persistence	User switches to dark mode	Preference stored in localStorage	PASS
TC22	Responsive Viewport Render	Viewport resized to 375px mobile	Sidebar collapses to drawer	PASS

CHAPTER 17 – SECURITY & DATA PRIVACY

Security and candidate confidentiality are paramount in ResumeCheck. Implemented measures include:

- Salted Hash Passwords: User passwords are encrypted with bcryptjs utilizing 10 salt rounds.
- Stateless JWT Verification: Protected routes enforce Bearer JWT token verification.
- SQL Injection Immunity: All queries to Microsoft SQL Server use parameterized inputs via mssql Request bindings.
- Input Sanitization: All request payloads are validated using express-validator before reaching business logic.
- Ephemeral PDF Ingestion: Uploaded resume PDFs are processed in volatile memory buffers without writing files to disk.

CHAPTER 18 – PERFORMANCE & ACCESSIBILITY

- Hardware-Accelerated Styling: Tailwind CSS utility classes and Framer Motion CSS transforms utilize GPU acceleration.
- Relational Indexing: Clustered and nonclustered indexes in SQL Server ensure sub-10ms query execution.
- Reduced Motion Support: Respects OS prefers-reduced-motion media query settings to disable animations.
- Shimmer Skeleton Loading: Prevents layout shifts during network data retrieval.

CHAPTER 19 – LIMITATIONS

- Keyword Heuristics: Keyword analysis measures vocabulary coverage but cannot evaluate deep narrative context.
- Dictionary Boundaries: Highly novel, proprietary, or misspelled technical terms may not be recognized.
- No Hiring Guarantee: Match percentage is an alignment indicator and cannot guarantee an employment interview.

CHAPTER 20 – FUTURE ENHANCEMENTS

- Semantic Vector Embeddings: Integrate dense vector representations to recognize semantic skill equivalents.
- DOCX and Rich Text Parsing: Extend file ingestion to Microsoft Word (.docx) documents.
- Sectional Resume Breakdown: Automatically parse Experience, Education, and Project sections individually.
- Downloadable PDF Reports: Enable candidates to export styled PDF analysis summary reports.
- Multi-Resume Comparison Matrix: Allow comparing multiple resumes against a single job specification.

CHAPTER 21 – CONCLUSION

The ResumeCheck project successfully demonstrates the design, engineering, and deployment of a full-stack resume keyword analysis web application. By combining React.js, Tailwind CSS, Framer Motion, Recharts, Node.js, Express.js, and Microsoft SQL Server, the application provides an objective, transparent, and aesthetically premium solution for job seekers.

Through rigorous testing across 22 test cases and real-world execution with PDF documents, ResumeCheck exhibits high accuracy in keyword matching, responsive performance, and secure data handling, fulfilling all project objectives.

REFERENCES

- React.js Official Documentation. Meta Platforms Inc. https://react.dev/
- Vite Next Generation Frontend Tooling. Evan You et al. https://vitejs.dev/
- Tailwind CSS Documentation. Tailwind Labs Inc. https://tailwindcss.com/
- Node.js Technical Documentation. OpenJS Foundation. https://nodejs.org/
- Express.js Application Framework. OpenJS Foundation. https://expressjs.com/
- Microsoft SQL Server Documentation. Microsoft Corporation. https://learn.microsoft.com/sql/
- Framer Motion API Reference. Motion Software Ltd. https://www.framer.com/motion/
- Recharts Composable Charting Library. https://recharts.org/

APPENDIX – IMPORTANT CODE SNIPPETS

Appendix A – Mathematical Scoring Formula (analysisService.js)

```
// Match calculation formula let matchPercentage = 0; if (totalJobKeywords > 0) {   matchPercentage = parseFloat(((matchedKeywordsList.length / totalJobKeywords) * 100).toFixed(2)); }  function getMatchTier(percentage) {   if (percentage >= 80) return "Excellent Keyword Match";   if (percentage >= 60) return "Good Keyword Match";   if (percentage >= 40) return "Moderate Keyword Match";   return "Low Keyword Match"; }
```

Appendix B – Boundary-Safe Keyword Regex (textProcessor.js)

```
// Safe boundary matching for technical phrases const escaped = escapeRegExp(trimmedCandidate); const pattern = new RegExp(`(^|[^a-zA-Z0-9_#\\+])${escaped}([^a-zA-Z0-9_#\\+]|$)`, "i"); if (pattern.test(normalized)) { matchedKeywordsMap.set(canonicalName, {     keyword: canonicalName,     category: item.category,     importance: item.importance   }); }
```

Appendix C – PDF Text Extraction (pdfExtractor.js)

```
// PDF parsing supporting modern Uint8Array buffer interface const uint8 = new Uint8Array(dataBuffer); const parser = new pdfModule.PDFParse(uint8); const result = await parser.getText(); const cleanText = result.text.replace(/-- \\d+ of \\d+ --/g, "").trim();
```